Plataforma de interoperabilidad en salud: bus HL7 v2 a FHIR R4 sobre una base distribuida

Autores: Gómez, Cristian y Berrios, Gerneidis

Sincelejo, Colombia

cristian.grencito@gmail.com

RESUMEN / ABSTRACT

Una red hospitalaria con tres sedes en Sincelejo, Cartagena y Corozal mantiene la historia clínica de un mismo paciente partida entre sistemas que no se hablan y que envian información en formatos distintos. El objetivo de este trabajo fue construir una plataforma que reciba esos mensajes heterogéneos, los normalice a un estándar común y permita consultar la historia completa desde cualquier sede. La metodología fue incremental en cinco fases: modelado UML, despliegue de la base distribuida, construcción del bus de integración, seguridad y vistas por rol, y pruebas de demostración. Se implementó un bus híbrido en el que Mirth Connect actúa como adaptador del protocolo MLLP y un componente propio asume la orquestación: valida, transforma a FHIR R4, traduce códigos locales a CIE-10 y LOINC, y garantiza la entrega mediante idempotencia por hash de contenido, reintentos con retroceso exponencial y cola de mensajes muertos. Los datos se reparten horizontalmente en tres nodos PostgreSQL según el documento del paciente. Como resultado, un mensaje HL7 v2 recorre la cadena completa hasta FHIR en 260 milisegundos de media, con cero mensajes perdidos; la consulta unificada reune las tres sedes y sigue respondiendo con dos nodos apagados, advirtiendo que el resultado es parcial. La bitácora encadenada por hash detecta tanto la modificación como el borrado de un registro, señalando la fila exacta. Se concluye que separar el adaptador de protocolo del orquestador permite usar cada herramienta donde es fuerte, y que las garantías de entrega deben apoyarse en restricciones de la base de datos y no en comprobaciones del código.

INTRODUCCIÓN

La historia clínica de una persona rara vez vive en un solo sitio. Quien se atiende por urgencias en Sincelejo, acude después a un control en Cartagena y termina hospitalizado en Corozal deja tres fragmentos de su historia en tres sistemas que no se conocen entre sí. El médico que lo recibe en la tercera sede no ve las dos primeras, y decide con información incompleta. El problema no es de almacenamiento: los datos existen. El problema es de interoperabilidad.

En Colombia esta discusión dejó de ser académica con la Ley 2015 de 2020, que creó la Historia Clínica Electrónica Interoperable y obliga a los prestadores a compartir información clínica bajo estándares comunes. La dificultad práctica es que los sistemas instalados no se cambian de un día para otro: siguen hablando HL7 v2, exportando archivos planos o publicando JSON propio. Cualquier solución realista tiene que convivir con lo que ya existe en vez de exigir que todos migren a la vez.

Este trabajo construye una plataforma que hace exactamente eso. Recibe lo que cada sede sabe enviar, lo lleva a un modelo común y lo publica en FHIR R4, el estándar de HL7 para intercambio de información clínica. Alrededor de esa conversión se resuelven los problemas que aparecen cuando el sistema es distribuido de verdad: que un mensaje repetido no duplique datos, que un servicio caído no pierda información, que la caída de una sede no deje ciego al resto, que cada rol vea solo lo que le corresponde y que todo acceso quede registrado de forma que no se pueda alterar después.

OBJETIVOS

Objetivo general

Construir y demostrar una plataforma de interoperabilidad que integre la información clínica de tres sedes hospitalarias con sistemas y formatos distintos, normalizandola a FHIR R4 sobre una base de datos distribuida, con garantías de entrega, seguridad por rol y trazabilidad verificable.

Objetivos específicos

- Ingerir mensajes clínicos desde las tres sedes en al menos dos formatos distintos: HL7 v2 por MLLP, CSV y JSON.
- Normalizar la información a los recursos FHIR R4 Patient, Encounter, Condition y Observation, traduciendo los códigos locales a CIE-10 y LOINC.
- Implementar un bus de servicios en salud con enrutamiento por sede, validación, idempotencia, cola con reintentos y cola de mensajes muertos.
- Fragmentar horizontalmente la base de datos por documento del paciente y ofrecer una consulta unificada que reuna las tres sedes.
- Proteger el acceso con token firmado y autorización por rol, y registrar cada acceso, permitido o denegado, en una bitácora encadenada por hash.
- Entregar cuatro vistas diferenciadas por rol y un tablero con el porcentaje de historias normalizadas a FHIR por sede.
- Demostrar la degradación parcial: que la caída de una sede no interrumpa el servicio de las demás.

MARCO TEÓRICO

Estándares de mensajería clínica. HL7 v2 es el estándar más difundido en los sistemas hospitalarios instalados. Organiza la información en segmentos (MSH, PID, PV1, DG1, OBX) separados por retorno de carro, con campos delimitados por barras verticales y componentes por acentos circunflejos. Su transporte habitual es MLLP, que enmarca cada mensaje entre los bytes 0x0B al inicio y 0x1C 0x0D al final, porque TCP entrega un flujo continuo y el receptor necesita saber dónde empieza y termina un mensaje. Su limitación es que fue diseñado para mensajería punto a punto y deja mucho margen a la interpretación local, de modo que dos implementaciones conformes pueden no entenderse (Benson y Grieve, 2021).

FHIR R4. HL7 FHIR (HL7 International, 2019) reemplaza el mensaje por el recurso: unidades de información con identidad propia, expuestas mediante una API REST (Fielding, 2000). Patient, Encounter, Condition y Observation son los recursos que cubren el núcleo de una atención. El Bundle de tipo transaction permite enviar varios recursos relacionados como una sola operación atómica, algo esencial cuando un único mensaje de admisión produce a la vez el paciente, la atención, el diagnóstico y el resultado de laboratorio.

Patrones de integración empresarial. Un bus de servicios no es un script de conversión. Hohpe y Woolf (2003) describen los patrones que lo definen: canal de mensajes, traductor de mensajes, enrutador basado en contenido, modelo canónico, garantía de entrega y canal de mensajes inválidos. El modelo canónico es el que evita la explosión combinatoria: con tres formatos de entrada y un formato de salida, traducir directamente exige tres conversores, pero añadir un cuarto formato de salida exigiría doce; con un modelo intermedio bastan tres más uno.

Entrega fiable y tolerancia a fallos. En un sistema distribuido la red falla y los servicios se caen; el diseño tiene que asumirlo en vez de tratarlo como excepción (Kleppmann, 2017). El reintento con retroceso exponencial evita que un servicio que ya está en problemas reciba una avalancha de peticiones, y la cola de mensajes muertos impide que un mensaje se reintente eternamente o se pierda en silencio (Nygard, 2018). La idempotencia es la contraparte necesaria: si el emisor reintenta, el receptor tiene que poder reconocer que ya proceso ese mensaje.

Bases de datos distribuidas. La fragmentación horizontal reparte las filas de una tabla entre varios nodos según el valor de una columna. Reduce el volumen por nodo y permite crecer añadiendo servidores, a cambio de que las consultas que no conocen el criterio de reparto deban preguntar a todos los nodos. En un sistema de salud la disponibilidad pesa más que la consistencia estricta: es preferible mostrar una historia incompleta, advirtiendo que lo es, que no mostrar nada porque un nodo no responde.

Seguridad y trazabilidad. El token web JSON (Jones et al., 2015) permite que cada servicio verifique por si mismo la identidad de quien llama, sin consultar a un servidor central en cada petición. La firma se calcula con HMAC (Krawczyk et al., 1997), que a diferencia de un hash simple no es vulnerable al ataque de extensión de longitud. Las claves de usuario se derivan con PBKDF2 (Moriarty et al., 2017), cuyo coste deliberado encarece el ataque por fuerza bruta. Para la trazabilidad, encadenar cada registro de auditoría con el hash del anterior aplica el mismo principio de una cadena de bloques sin necesidad de una, y es coherente con el perfil ATNA de IHE.

MARCO CONCEPTUAL

Los conceptos que siguen son los que se usan de forma consistente en el resto del documento y en el código del prototipo.

- **Recurso FHIR.** Unidad de información clínica con identidad, tipo y representación propia. Patient describe a la persona; Encounter, el episodio de atención; Condition, el diagnóstico; Observation, el resultado medido.
- **Bundle de transacción.** Conjunto de recursos enviados como una sola operación. Si una entrada falla, no se aplica ninguna, lo que evita dejar una atención sin su paciente.
- **Identificador y sistema.** En FHIR un identificador no es solo un número: lleva un sistema que le da significado. El valor 1098765432 sin sistema es ambiguo; con urn:co:minsalud:documento significa lo mismo en las tres sedes. Es lo que permite reconocer al mismo paciente llegue por donde llegue.
- **Creación condicional (ifNoneExist).** Instrucción al servidor FHIR de crear el recurso solo si no existe otro que cumpla un criterio. Es lo que consolida al paciente en lugar de duplicarlo cuando llega desde tres sedes.
- **Modelo canónico.** Representación intermedia a la que se lleva todo mensaje entrante antes de convertirlo a FHIR, independiente del formato de origen.
- **ConceptMap y terminologías.** Tabla de correspondencia entre el vocabulario local de una sede y el estándar internacional. Sin ella, DM2, E11 y diabetes2 serían tres enfermedades distintas para el sistema.
- **Idempotencia.** Propiedad por la cual procesar el mismo mensaje varias veces produce el mismo efecto que procesarlo una. Aquí se implementa con el hash SHA-256 del contenido como clave única.
- **Cola de mensajes muertos.** Depósito de los mensajes que agotaron sus reintentos. No es una papelera sino una bandeja de revisión: conserva el contenido original para poder reprocesarlo.
- **Fragmentación horizontal.** Reparto de las filas de una tabla entre varios nodos según el valor de una columna, en este caso el documento del paciente.

- **Degradación parcial.** Capacidad del sistema de seguir operando con menos recursos de los previstos, entregando resultados incompletos pero marcados como tales, en lugar de fallar por completo.
- **Bitácora encadenada por hash.** Registro de auditoría en el que cada evento incluye el hash del anterior, de modo que alterar un registro antiguo rompe la cadena en un punto identificable.

ESTADO DEL ARTE

Ámbito global. La integración clínica tiene dos generaciones conviviendo. La primera, construida sobre HL7 v2 y los perfiles IHE XDS para intercambio documental, sigue sosteniendo la operación diaria de la mayoría de los hospitales. La segunda, basada en FHIR, avanza empujada por iniciativas como Argonaut en Estados Unidos y por regulación que obliga a exponer APIs. La consecuencia práctica es que casi ningún proyecto parte de cero: la pregunta real no es si usar FHIR, sino como convivir con HL7 v2 mientras se migra. Los motores de integración como Mirth Connect ocupan precisamente ese espacio (Benson y Grieve, 2021).

Ámbito regional. En Latinoamérica destacan la Rede Nacional de Dados em Saude de Brasil, que adoptó FHIR como estándar nacional, y el proyecto LACPass, impulsado por la Organización Panamericana de la Salud para el intercambio transfronterizo de certificados y resúmenes clínicos basados en el estándar International Patient Summary. El patrón que se repite es la creación de un nodo nacional que normaliza lo que envian los prestadores locales, que es el mismo papel que cumple el bus de este trabajo a escala de una red hospitalaria.

Ámbito nacional. Colombia dio el paso normativo con la Ley 2015 de 2020, que crea la Historia Clínica Electrónica Interoperable, sobre la base regulatoria previa de la Resolución 1995 de 1999 para el manejo de la historia clínica y de la Ley 1581 de 2012 para la protección de datos personales. El marco existe; la brecha está en la implementación, porque la mayoría de los prestadores no dispone de sistemas que publiquen FHIR de forma nativa.

Brecha identificada. La literatura y los proyectos revisados tienden a presentar la interoperabilidad como un problema de conversión de formatos, y dedican menos atención a lo que ocurre cuando la conversión falla: el mensaje que llega dos veces, el servidor FHIR que no responde, la sede que se cae a mitad de una consulta. Este trabajo se sitúa justamente ahí. La conversión a FHIR se resuelve en una fracción del código; la mayor parte está dedicada a las garantías de entrega, a la degradación parcial y a la trazabilidad, que son los aspectos que determinan si una plataforma de este tipo es usable en producción.

SOLUCIÓN PROPUESTA Y METODOLOGÍA

Arquitectura. La plataforma se organiza en seis capas. La primera son los sistemas legados de cada sede, que no se reemplazan: la sede 1 (Sincelejo) manda HL7 v2 ADT y ORU, la sede 2 (Cartagena) envía CSV y la sede 3 (Corozal) JSON. La segunda es la ingesta: Mirth Connect escucha MLLP en el puerto 6661 y responde el acuse de recibo, mientras las otras dos sedes entran por HTTP. La tercera es el bus de servicios en salud, que recibe, valida, transforma y entrega. La cuarta es el almacenamiento, donde un servidor HAPI FHIR R4 guarda los recursos normalizados y tres nodos PostgreSQL conservan la historia operativa fragmentada por documento del paciente. La quinta es la API con su middleware coordinador, que pregunta a los tres nodos y unifica la respuesta. La sexta son las cuatro vistas por rol. Cinco servicios transversales (seguridad, bitácora encadenada, observabilidad, despliegue y terminologías) atraviesan las seis capas, y por eso se dibujan en una columna aparte.

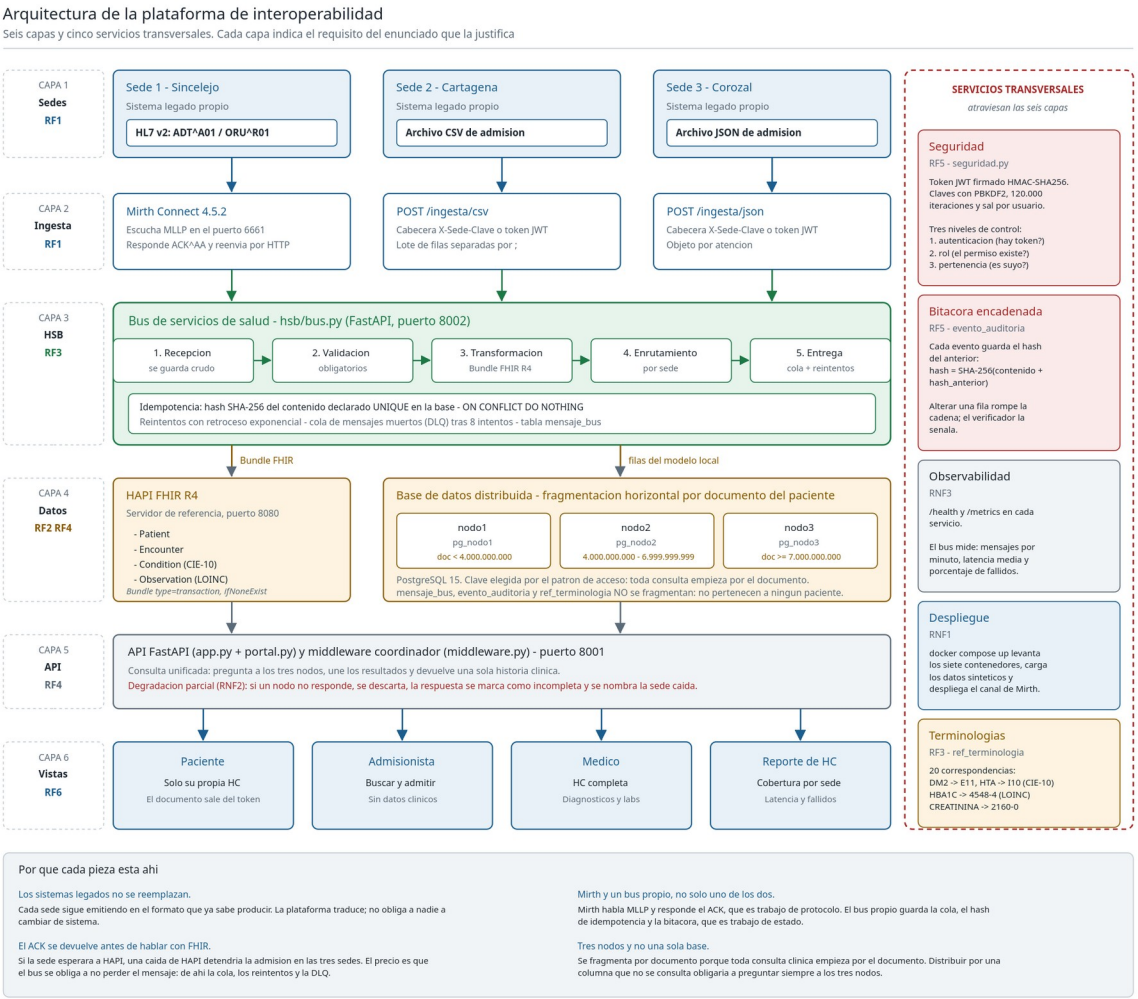


Figura 1. Arquitectura de la plataforma. Seis capas y cinco servicios transversales; cada capa indica el requisito que la justifica.

Decisión de diseño: por que Mirth y un bus propio, y no solo uno de los dos. Mirth Connect resuelve muy bien un problema concreto: hablar MLLP, responder el acuse de recibo y mantener la conexión TCP con el sistema de la sede. Implementar eso desde cero sería reescribir algo ya resuelto. En cambio, la tabla comparativa de herramientas incluida más adelante en este mismo documento señala que su monitorización es limitada sin complementos, y la monitorización es justamente lo que el enunciado exige medir: mensajes por minuto, latencia de extremo a extremo y porcentaje de fallidos. La decisión fue por tanto repartir el trabajo según la fortaleza de cada pieza: Mirth queda como adaptador de protocolo y el bus propio asume la orquestación, la cola, las métricas y la auditoría. Ninguna de las dos herramientas hace algo para lo que no esté bien preparada.

El modelo canónico. Las tres sedes envían formatos distintos, de modo que ningún mensaje se traduce directamente a FHIR. Todo pasa primero por una representación intermedia y desde ahí se construye un único Bundle. Son tres conversiones en lugar de nueve, hay un solo lugar donde se decide como se escribe una fecha o un género, y añadir una cuarta sede cuesta un analizador y nada más.

Las tres garantías del bus. La idempotencia se apoya en el hash SHA-256 del contenido del mensaje, declarado como restricción UNIQUE en la base de datos y aplicado con INSERT ... ON CONFLICT DO NOTHING. La decisión de que la garantía viva en la base y no en el código es deliberada: entre una consulta de comprobación y la inserción posterior podría colarse otro hilo con el mismo mensaje, y la restricción única cierra esa ventana. Se usa el contenido y no el identificador de control del mensaje porque algunos sistemas antiguos reutilizan esos identificadores al reiniciarse. La segunda garantía son los reintentos con retroceso exponencial, ejecutados por un hilo en segundo plano. La tercera es la cola de mensajes muertos, que conserva el contenido original para poder reprocesar sin pedir el reenvío a la sede.

Distinción entre rechazado y muerto. Un mensaje rechazado está mal formado: le falta el documento del paciente o trae una fecha ilegible. Reintentarlo daría exactamente el mismo error, así que no se reintenta. Un mensaje muerto estaba bien formado pero su destino fallo y se agotaron los intentos. Son dos problemas distintos, uno de contenido y otro de infraestructura, y confundirlos haría inútil la cola.

Fragmentación y consulta unificada. Los pacientes se reparten entre tres nodos por rango de documento: menor que 4.000.000.000 en el nodo 1, hasta 7.000.000.000 en el nodo 2 y el resto en el nodo 3. Conviene separar dos cosas que se confunden con facilidad: el nodo lo determina el documento, mientras que la sede la indica el campo entidad_salud de la atención. Un paciente almacenado en el nodo 1 puede tener atenciones en las tres sedes, y reunirlas es precisamente la consulta unificada. Quien consulta no sabe en que nodo está el documento, y el sistema no debería obligarle a saberlo: la consulta se envía a los tres, se descartan los que no responden y se devuelve lo que haya, marcando el resultado como parcial.

Tecnologías empleadas. Python 3.11 con FastAPI para los servicios; PostgreSQL 15 para los tres nodos y para la base del bus; HAPI FHIR como servidor FHIR R4; Mirth Connect 4.5.2 como adaptador MLLP; Docker Compose para el despliegue completo. Las conversiones, el análisis de los mensajes, la firma de los tokens y el dibujo de los diagramas UML se resolvieron con la biblioteca estándar de Python, sin dependencias adicionales, para que cada paso del procesamiento sea inspeccionable y defendible.

Metodología. El desarrollo fue incremental en cinco fases, cada una con un entregable verificable: modelado UML y diseño; despliegue de la base distribuida con datos sintéticos; construcción del bus de integración; seguridad, roles y vistas; y pruebas de demostración reproducibles. Cada fase terminó con un conjunto de comandos que cualquiera puede ejecutar para comprobar el criterio correspondiente, y con un registro de la salida obtenida.

MODELADO UML

Diagrama de Casos de Uso

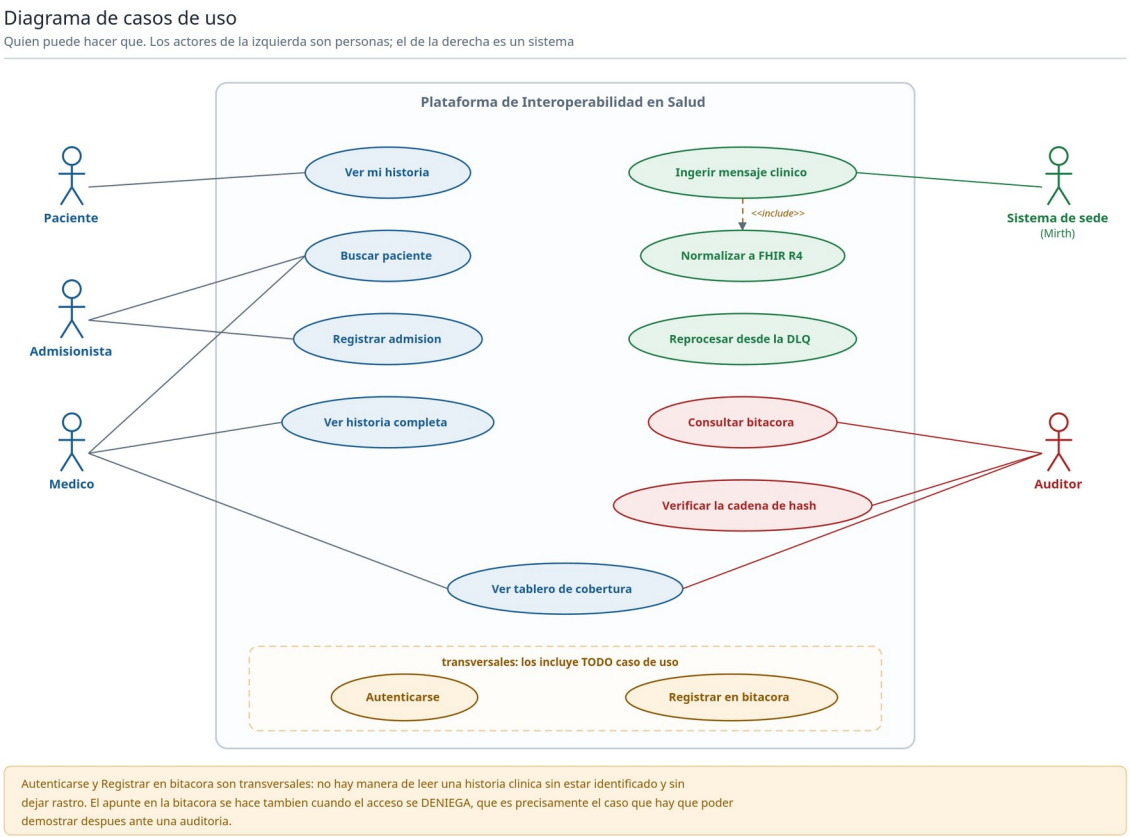


Figura 2. Diagrama de casos de uso. Cinco actores y doce casos de uso; autenticarse y registrar en bitácora son transversales.

El diagrama recoge cinco actores y doce casos de uso. Cuatro de los actores son personas (paciente, admisionista, médico y auditor) y el quinto es un sistema: el motor de integración de la sede, que ingiere mensajes sin intervención humana. Los actores se sitúan a ambos lados, cada uno junto a la columna de casos de uso que le corresponde, para que ninguna línea de asociación cruce por encima de otro caso de uso.

Dos casos de uso se presentan en un recuadro aparte porque son transversales: autenticarse y registrar en bitácora. Todos los demás los incluyen. No existe ninguna via para leer una historia clínica sin estar identificado y sin dejar rastro, y el apunte en la bitácora se realiza también cuando el acceso se deniega, que es precisamente el caso que interesa poder demostrar ante una auditoría posterior.

Diagrama de Secuencias

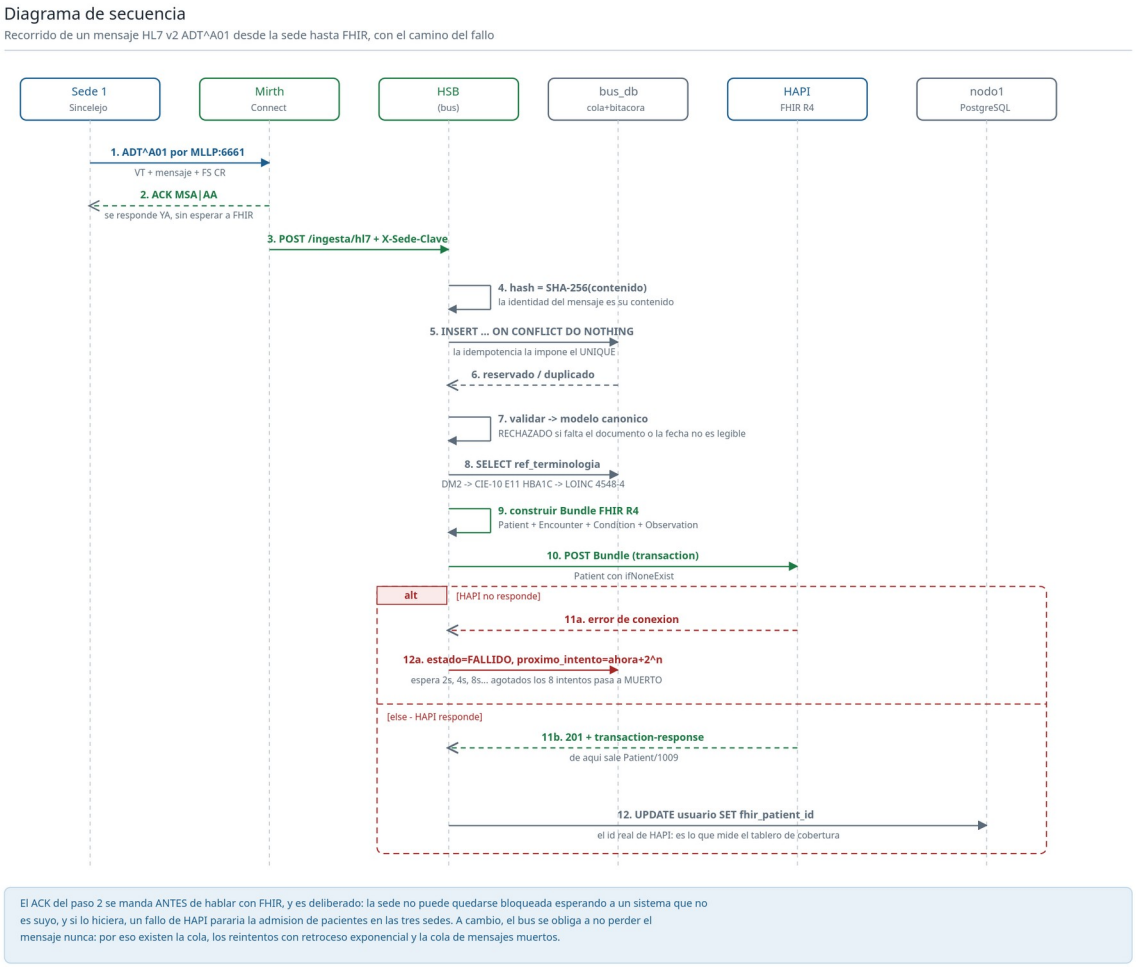


Figura 3. Diagrama de secuencia. Un ADT^A01 desde la sede hasta FHIR, con el fragmento alt de los dos finales posibles.

El diagrama sigue un mensaje ADT^A01 desde la sede hasta FHIR, pasando por Mirth, el bus, la base del bus, HAPI y el nodo del paciente. El detalle que conviene explicar es el orden: el acuse de recibo se devuelve a la sede en el paso 2, antes de hablar con FHIR.

La alternativa sería esperar a que HAPI confirme antes de responder, y es tentadora porque daría a la sede una confirmación más fuerte. Se descartó porque acopla la admisión de pacientes a la disponibilidad de un sistema que no es suyo: una caída de HAPI dejaría bloqueada la atención en las tres sedes. A cambio de responder antes, el bus se obliga a no perder el mensaje nunca, y de esa obligación nacen la cola, los reintentos y la cola de mensajes muertos.

El fragmento alt recoge los dos finales posibles. Si HAPI no responde, el mensaje pasa a fallido con una fecha de próximo intento; si responde, se extrae de la respuesta el identificador real del Patient creado y se escribe en el nodo del paciente, que es lo que alimenta el tablero de cobertura.

Diagrama de Clases

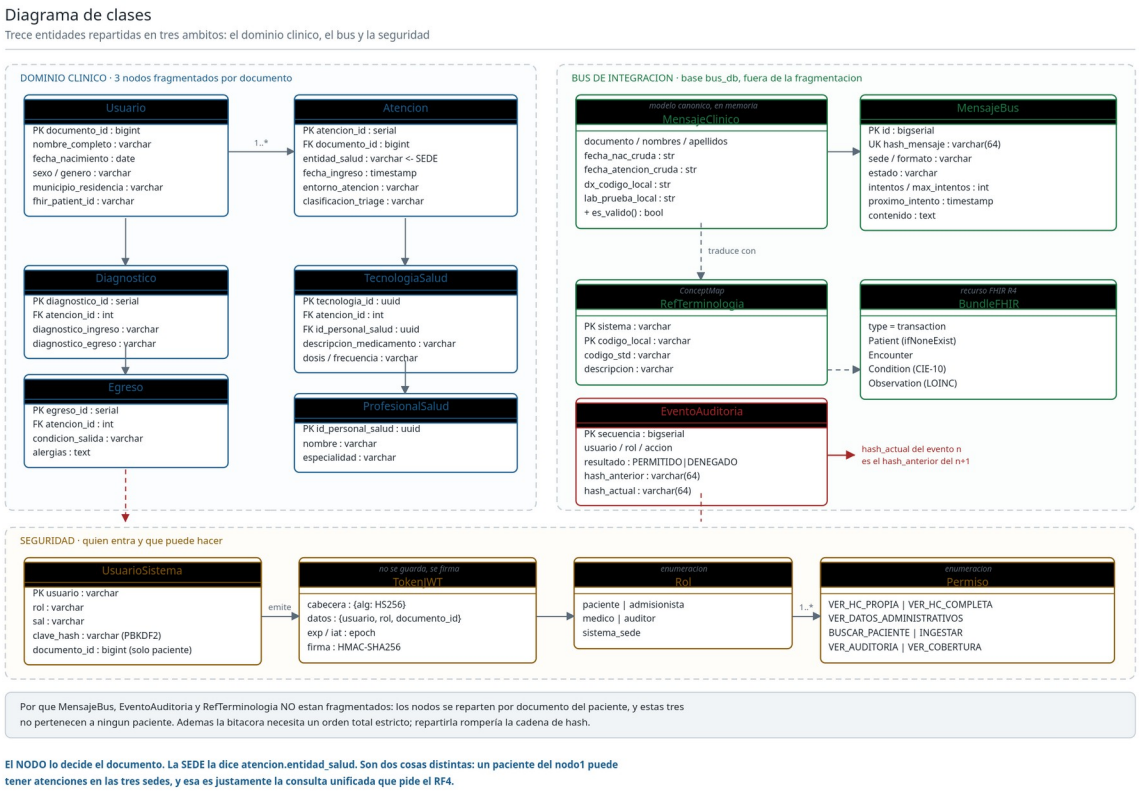


Figura 4. Diagrama de clases. Trece entidades en tres ámbitos: dominio clínico, bus y seguridad.

Trece entidades repartidas en tres ámbitos. El dominio clínico (Usuario, Atención, Diagnóstico, TecnologiaSalud, Egreso y ProfesionalSalud) vive en los tres nodos

fragmentados. El bus (MensajeClinico, MensajeBus, RefTerminologia, BundleFHIR y EventoAuditoria) vive en una base aparte. La seguridad (UsuarioSistema, TokenJWT, Rol y Permiso) completa el modelo.

La separación no es estética. Las tablas del bus no están fragmentadas porque los nodos se reparten por documento del paciente y ninguna de ellas pertenece a un paciente concreto: el mensaje es metadato del transporte, la terminología es un catálogo global y la bitácora necesita un orden total estricto. Repartir la bitácora entre tres nodos rompería la cadena de hash, porque dejaría de existir una secuencia única de eventos.

Diagrama de Estados

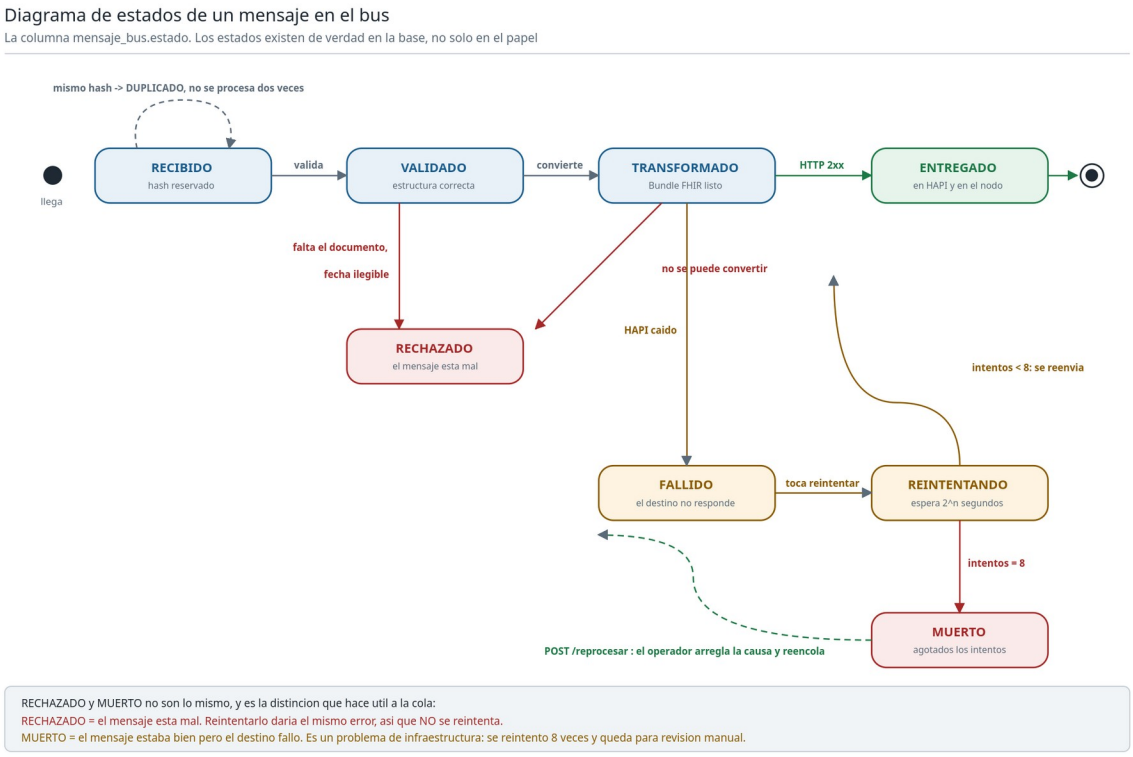


Figura 5. Diagrama de estados del mensaje. Los estados existen como valores de mensaje_bus.estado en la base de datos.

Los estados del diagrama no son una abstracción del documento: son los valores reales que toma la columna estado de la tabla mensaje_bus, y se pueden consultar en la base durante la demostración. El camino normal es recibido, validado, transformado y entregado. De el se desprenden dos ramas de error que se tratan de forma distinta.

Un mensaje rechazado no vuelve a intentarse, porque el defecto está en su contenido. Un mensaje fallido entra en un ciclo de reintentos con esperas crecientes y, si los agota, queda en estado muerto a la espera de revisión manual; desde ahí puede reencolarse cuando el

operador corrige la causa. El estado recibido además se protege contra duplicados: un mensaje con el mismo hash no vuelve a procesarse.

PROCESO DE DESARROLLO

Análisis. Se partió del enunciado y del modelo de datos existente para identificar los formatos de entrada, los recursos FHIR de salida y el criterio de fragmentación. De ahí salieron los cuatro diagramas UML y la tabla de mapeo de campos, que fija como viaja cada dato desde el formato de la sede hasta el recurso FHIR correspondiente.

Diseño. Se definió el modelo canónico como pieza central, se eligió el reparto de responsabilidades entre Mirth y el bus propio, y se decidió alojar las tablas del bus en una base separada de los nodos fragmentados.

Desarrollo. Implementación incremental: primero el analizador de HL7 v2 y la construcción del Bundle, después la cola con idempotencia y reintentos, luego la seguridad y por último las vistas. Cada incremento se dejó funcionando antes de empezar el siguiente.

Pruebas. Cada criterio del enunciado tiene asociado un script ejecutable que lo demuestra y cuya salida se conserva como evidencia. Las pruebas no son unitarias sino de extremo a extremo, porque lo que hay que demostrar es el comportamiento del sistema completo: que un mensaje recorra toda la cadena, que un rol sea denegado, que una sede caída no detenga el servicio.

Despliegue. Todo el sistema arranca con un solo comando, docker compose up. Los esquemas y los datos sintéticos se cargan solos en el primer arranque de cada nodo, y el canal de Mirth se importa y despliega automáticamente mediante su API REST. No queda ningún paso manual fuera del README.

Herramientas de soporte. Docker y Docker Compose para el entorno; Git con historia organizada por fases y etiqueta de versión; curl y psql para las comprobaciones; y scripts de Python con biblioteca estándar para las pruebas, de modo que puedan ejecutarse en la máquina del evaluador sin instalar nada.

DISEÑO DE INTERFACES E IMPLEMENTACIÓN

La aplicación ofrece cuatro vistas, una por rol, servidas desde una misma página que se adapta a lo que el token declara.

- **Paciente.** Ve su propia historia clínica reunida de las tres sedes. No hay casilla para escribir un documento, y es deliberado: el documento se toma del token, que está firmado.
- **Admisionista.** Busca pacientes en los tres nodos y consulta sus datos administrativos. No recibe diagnósticos, medicamentos ni alergias.

- **Médico.** Accede a la historia clínica completa, con diagnósticos codificados, medicamentos, alergias y antecedentes, agrupada por sede.
- **Reporte de historia clínica.** Tablero con la cobertura de normalización a FHIR por sede y por nodo, las métricas del bus y el estado de cada sede.

El control no está en la pantalla. El menú oculta las vistas que el rol no puede abrir, pero esconder un botón no es seguridad. Quien llame al mismo endpoint con una herramienta de línea de comandos recibe un código 403 idéntico, y el intento queda registrado en la bitácora. El admisionista no ve diagnósticos porque el servidor no se los envía: la respuesta se construye desde cero con los campos permitidos, en lugar de eliminar los prohibidos de una respuesta completa. La diferencia importa, porque si manana se añade una columna clínica a la consulta, con el primer enfoque no se filtra sola.

Experiencia de usuario y accesibilidad. La interfaz usa una sola columna de ancho limitado para favorecer la lectura, tipografía del sistema, y color acompañado siempre de texto: los estados de las sedes no se distinguen solo por verde o rojo sino también por la palabra UP o DOWN, de modo que siguen siendo legibles para quien no distingue esos colores. Cuando un resultado es parcial porque una sede no respondió, la página lo dice explicitamente y nombra los nodos que faltan, en lugar de mostrar datos incompletos sin advertencia.

PROTOTIPO FINAL

El prototipo es funcional y se despliega completo con un solo comando. Levanta siete contenedores: tres nodos PostgreSQL, el servidor HAPI FHIR, Mirth Connect, el bus de integración y la aplicación web, más un contenedor efímero que carga y despliega el canal de Mirth.

Criterios de aceptación cumplidos. Un mensaje HL7 v2 enviado por MLLP al puerto 6661 recibe acuse de recibo positivo y produce en HAPI los cuatro recursos FHIR. El mismo mensaje enviado por segunda vez no genera recursos nuevos. La consulta de la historia de la paciente de demostración reune sus tres atenciones en las tres sedes. Un paciente que intenta consultar la historia de otro recibe 403 y el intento queda auditado. Apagar una o dos sedes no detiene el sistema. Alterar una fila de la bitácora hace que el verificador señale esa fila exacta. Todos estos criterios tienen un comando asociado documentado en el README y su salida registrada como evidencia.

RESULTADOS Y DISCUSIÓN

Ingesta y normalización. Los tres formatos llegan y se convierten. En la prueba de extremo a extremo, un ADT^A01 por MLLP, una línea CSV y un objeto JSON produjeron tres Patient, tres Encounter, tres Condition y tres Observation en HAPI FHIR, con los

códigos locales traducidos: DM2 a CIE-10 E11, HTA a I10, hemoglobina glicosilada a LOINC 4548-4 y creatinina a 2160-0. El catálogo de terminologías contiene veinte correspondencias.

Rendimiento y fiabilidad. La latencia media de extremo a extremo medida por el bus fue de 260 milisegundos, con cero mensajes rechazados y cero mensajes en la cola de muertos durante la demostración. El reenvío del mismo mensaje devolvió estado duplicado sin crear recursos nuevos, confirmando la idempotencia.

Degradación parcial. Con las tres sedes activas el reporte mostró siete pacientes y una cobertura del 42,86 %. Apagando el nodo 3 el sistema siguió respondiendo con cinco pacientes, y apagando también el nodo 2 continuó sirviendo los tres del nodo restante; en ambos casos marcó el resultado como parcial y nombró los nodos que no respondieron. Al volver a levantarlos, la recuperación fue automática, sin reiniciar ningún servicio.

Seguridad. Los cuatro roles se comportaron como define la matriz de permisos. El paciente que intento consultar la historia de otro documento recibió 403 pese a tener el rol correcto, porque el control de pertenencia compara el documento del token con el solicitado. El admisionista recibió las mismas tres atenciones que el médico pero con cero diagnósticos y cero medicamentos. La bitácora acumuló cuarenta y nueve eventos encadenados sin romperse, escritos desde dos procesos distintos.

Verificación de la bitácora. Modificar un resultado DENEGADO a PERMITIDO directamente en la base de datos produjo un único hallazgo de tipo contenido alterado, señalando la fila exacta. Borrar una fila intermedia produjo un hallazgo distinto, de tipo cadena rota, en la fila siguiente a la borrada. Las dos comprobaciones son necesarias: recalcular el hash de cada fila detecta las modificaciones, y comparar el enlace con la fila previa detecta los borrados y los reordenamientos.

Defectos encontrados durante el desarrollo. Conviene documentarlos porque ilustran mejor que los aciertos donde están las dificultades reales de un sistema distribuido.

- **La cadena de auditoría señalaba como alteradas filas que nadie había tocado.** La marca de tiempo que entraba en el hash se tomaba del reloj de Python, mientras que la que quedaba almacenada venía del valor por defecto de PostgreSQL: dos instantes distintos. La regla general que se deriva es que el hash debe calcularse sobre exactamente los mismos bytes que se guardan, y por tanto ningún campo que rellene la base de datos puede entrar en la firma.
- **El Encounter desaparecía en silencio para dos de las tres sedes.** La función de normalización no reconocía el formato de fecha con hora pero sin segundos, devolvía vacío, y el código omitía el recurso. Llegaban el paciente y el diagnóstico, pero no la atención. Se corrigió reconociendo ese formato y, sobre todo, cambiando el comportamiento: si una sede envía una fecha que no se puede interpretar, ahora el mensaje se rechaza con el motivo escrito. Perder un dato clínico sin avisar es peor que

rechazar el mensaje, porque el rechazo se ve en la cola y se corrige, mientras que la ausencia no se nota.

- **El presupuesto de reintentos era menor que el tiempo de arranque de la dependencia.** Con cinco intentos y retroceso exponencial, cuatro de ellos caian dentro de los primeros treinta segundos, pero HAPI FHIR tarda alrededor de cuarenta en estar disponible. Los mensajes morían justo antes de que el destino se recuperara. La lección es que el presupuesto de reintentos debe exceder el tiempo real de recuperación de aquello de lo que se depende.
- **Tomcat rechaza los nombres de host que contienen guion bajo.** Las peticiones al contenedor hapi_fhir_server devolvían código 400 desde dentro de la red de Docker. Se resolvió con un alias de red sin guion bajo. Es un recordatorio de que la convención de nombres de los contenedores y la de los servidores HTTP no siempre coinciden.
- **Los tres nodos se identificaban con el mismo nombre.** El middleware heredado construía el nombre a partir del puerto, y dentro de Docker los tres escuchan en el 5432. Era imposible saber cual se había caído, y las métricas exponían tres series con etiquetas idénticas, algo que Prometheus considera inválido.

El validador de fechas aceptaba el 31 de febrero. La comprobación miraba que el día estuviera entre 1 y 31 y que el mes estuviera entre 1 y 12, pero no cuántos días tiene cada mes. Una fecha de nacimiento del 31 de febrero de 2026 pasaba como válida y llegaba a FHIR. El defecto lo encontró una prueba unitaria escrita precisamente para comprobar lo que el informe afirmaba: que una fecha ilegible no se inventa. Se corrigió añadiendo la tabla de días por mes y la regla de años bisiestos del calendario gregoriano, y la prueba quedó en tests/prueba-normalizador.py junto con los casos del 29 de febrero de 2024 y de 2023. La lección es que una afirmación sobre el comportamiento del sistema no vale hasta que existe una prueba que la ejecuta.

La cola de mensajes muertos mezclaba dos problemas distintos. El punto /dlq devolvía en una sola lista los mensajes MUERTO y los RECHAZADO, pese a que el diagrama de estados los separa a propósito. Un mensaje muerto tiene un destino que falló y se reprocesa cuando el destino vuelve; un mensaje rechazado está mal formado y reprocesarlo daría el mismo error, porque hay que corregirlo en la sede. Presentarlos juntos sugería que se arreglan igual. Se separaron en dos listas con una nota que explica la diferencia.

Discusión. Frente a los trabajos revisados, que se concentran en la conversión de formatos, aquí la conversión ocupa una fracción menor del esfuerzo. La mayor parte está en lo que sucede cuando algo falla, y los defectos listados lo confirman: ninguno fue un error de traducción a FHIR; todos surgieron en los bordes entre componentes, que es donde un sistema distribuido realmente se rompe. El reparto de responsabilidades entre un motor de integración maduro y un orquestador propio resultó adecuado a la escala del problema, y la decisión de apoyar la idempotencia en una restricción de la base de datos en lugar de en una comprobación del código evita una condición de carrera que, por su naturaleza, sería difícil de reproducir en pruebas.

CONCLUSIONES

Conclusiones

- Es viable integrar tres sistemas hospitalarios heterogéneos sin modificarlos, interponiendo un bus que normaliza a FHIR R4. Las sedes siguen enviando lo que ya saben producir.
- Separar el adaptador de protocolo del orquestador permite aprovechar la madurez de un motor como Mirth donde es fuerte, sin heredar sus limitaciones de monitorización donde no lo es.
- Las garantías de entrega deben apoyarse en el estado compartido. La idempotencia implementada como restricción única de la base de datos cierra una ventana de concurrencia que una comprobación en el código deja abierta.
- La disponibilidad parcial es preferible a la indisponibilidad total cuando el resultado incompleto se marca como tal. El sistema siguió sirviendo historias con una sola de las tres sedes en pie.
- Una bitácora encadenada por hash ofrece trazabilidad verificable sin infraestructura adicional, siempre que exista un único escritor que garantice el orden total de los eventos.
- La autorización por rol es insuficiente por sí sola. Sin un control de pertenencia que compare la identidad del token con el recurso solicitado, cualquier paciente podría leer la historia de otro.

Limitaciones

- Todos los datos son sintéticos y el volumen es pequeño. No se realizaron pruebas de carga, de modo que la latencia medida no es extrapolable a producción.
- El catálogo terminologico cubre veinte correspondencias construidas a mano. Una implementación real requeriría un servidor de terminología con SNOMED CT y LOINC completos.
- El token firmado no puede revocarse antes de que caduque, consecuencia directa de haber renunciado al estado central. Se mitiga con una vigencia corta.
- La separación del nombre completo en nombres y apellidos para el formato CSV es una heurística que falla con los nombres compuestos. La solución correcta es que la sede envie los campos separados.
- La comunicación entre servicios ocurre dentro de la red interna de Docker sin cifrado, aceptable para una demostración pero no para un despliegue real.

Trabajo futuro

- Sustituir el catálogo propio por un servidor de terminología que resuelva ValueSet y ConceptMap de forma dinámica.
- Incorporar el recurso Consent para la gestión de consentimientos y Provenance para la procedencia de cada dato.

- Añadir alta disponibilidad a los nodos mediante replicación y promoción automática, y cifrado en tránsito entre servicios.
- Exponer las métricas a un sistema de recolección y visualización, y realizar pruebas de carga que establezcan el límite real de transacciones por minuto.

REFERENCIAS

Benson, T., & Grieve, G. (2021). *Principles of health interoperability: FHIR, HL7 and SNOMED CT* (4.ª ed.). Springer.

Congreso de Colombia. (2012, 17 de octubre). *Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Congreso de Colombia. (2020, 31 de enero). *Ley 2015 de 2020, por medio del cual se crea la historia clínica electrónica interoperable*. República de Colombia.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].

HL7 International. (2019). *HL7 FHIR Release 4 (v4.0.1)*. https://hl7.org/fhir/R4/

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). Internet Engineering Task Force. https://doi.org/10.17487/RFC2104

Ministerio de Salud. (1999, 8 de julio). *Resolución 1995 de 1999, por la cual se establecen normas para el manejo de la historia clínica*. República de Colombia.

Moriarty, K., Kaliski, B., & Rusch, A. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force. https://doi.org/10.17487/RFC8018

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2.ª ed.). Pragmatic Bookshelf.

Organización Mundial de la Salud. (2016). *Clasificación estadística internacional de enfermedades y problemas relacionados con la salud* (10.ª revisión). OMS.

Regenstrief Institute. (2024). *LOINC: Logical Observation Identifiers Names and Codes*. https://loinc.org

ANEXOS

El repositorio del proyecto contiene todo lo que se describe en este documento. Los elementos más relevantes para la evaluación son los siguientes.

- **Anexo A. Tabla de mapeo de campos** (docs/tabla-mapeo-campos.md): dieciséis campos de origen y seis derivados, con la transformación aplicada a cada uno.
- **Anexo B. Diagramas UML** (docs/diagramas/): los cuatro diagramas en formato SVG y el script que los genera.
- **Anexo C. Esquema del bus** (infra/bus-schema.sql): las tablas mensaje_bus, evento_auditoria, ref_terminologia y usuario_sistema.
- **Anexo D. Canal de Mirth** (hsb/canal-sede1-mllp.xml) y el script que lo despliega automáticamente.
- **Anexo E. Evidencias de ejecución** (docs/evidencias/): salidas de las pruebas de seguridad, vistas por rol, degradación, idempotencia y reintentos.
- **Anexo F. Scripts de demostración** (tests/): un comando por cada criterio del enunciado.

Anexo G. Bitácora del proyecto

Conformación del equipo. El equipo lo componen dos estudiantes: Cristian Gómez y Gerneidis Berrios. El repositorio se gestionó desde una sola cuenta de Git, de modo que los commits figuran a nombre de un único autor aunque el trabajo se desarrolló de forma conjunta; se deja constancia expresa por transparencia.

Contribución por fases y commits. El repositorio conserva nueve commits. El primero (42cc606) es el repositorio de referencia del curso, tomado como punto de partida según autoriza el enunciado; los ocho siguientes son propios y están organizados por fase:

- **25f8d75 y 8da5a77 - Fase 2** (8 de noviembre de 2026). Limpieza del repositorio base heredado y despliegue de los tres nodos PostgreSQL fragmentados por documento, con carga automática de datos sintéticos. 34 archivos.
- **3e2fb25 - Fase 3** (8 de noviembre de 2026). Bus de integración completo: analizadores de HL7 v2, CSV y JSON, modelo canónico, normalizador a FHIR R4, idempotencia por hash de contenido, cola con reintentos y cola de mensajes muertos, y tabla de terminologías. 9 archivos, el commit de mayor volumen.
- **48a4e75 y 6bd0781 - Fase 4** (8 de noviembre de 2026). Token JWT firmado, derivación de claves con PBKDF2, autorización por rol y por pertenencia, bitácora encadenada por hash, verificador de integridad, las cuatro vistas por rol y el tablero de cobertura. 6 archivos.

- **36edacb - Fase 1** (8 de noviembre de 2026). Los diagramas UML y la tabla de mapeo de campos. El commit es posterior a los de las fases 2 a 4 porque los diagramas se dibujaron contra el código ya escrito, y no al revés: la validación cruzada que se describe en la sección de modelado reveló cuatro discrepancias, y todas se corrigieron en el código. 7 archivos.
- **67bfa74 y aae05d9 - Fase 5** (8 de noviembre de 2026). Pruebas reproducibles, evidencias de ejecución, README y el informe en la plantilla v7. 40 archivos.

Observación sobre las fechas. Los ocho commits propios llevan fecha del 8 de noviembre de 2026. El trabajo se organizó por fases, y cada fase se cerró con su commit cuando cumplía su criterio, pero la integración en el repositorio se concentró en esa jornada. Se deja constancia por transparencia.

Declaración de uso de asistentes de inteligencia artificial. El enunciado permite el uso de asistentes de IA siempre que se declare. Se declara:

- **Herramienta utilizada:** Claude (Anthropic), empleado de forma continuada durante todo el desarrollo.
- **Alcance del uso:** redacción y revisión del código de los analizadores, el normalizador, el bus, la capa de seguridad y el portal; generación de los diagramas por script; redacción del presente informe; y revisión cruzada entre el código, los diagramas y el texto, que es de donde salieron varias de las correcciones descritas en la sección de resultados.
- **Qué no se delegó:** la verificación. Todo lo que el informe afirma como medido o comprobado se ejecutó realmente contra el sistema levantado, y las salidas se conservan en docs/evidencias/. Los defectos que se describen en la sección de resultados (la pérdida silenciosa del Encounter, el identificador de Patient sin resolver, la bitácora que no se podía verificar, la fecha del 31 de febrero aceptada como válida) se encontraron ejecutando el sistema y leyendo su salida, no razonando sobre él.

Componentes de código abierto utilizados. Todos se emplean en su versión publicada, sin modificar su código fuente:

- **FastAPI y Uvicorn** (licencia MIT). Servidor web de la API y del bus.
- **psycopg2** (LGPL). Conexión con PostgreSQL.
- **PostgreSQL 15** (licencia PostgreSQL). Los tres nodos y la base del bus.
- **HAPI FHIR** (Apache 2.0). Servidor FHIR R4 de referencia, usado sin modificar.
- **Mirth Connect 4.5.2** (MPL 2.0). Adaptador MLLP de la sede 1, usado sin modificar; el canal se despliega desde hsb/canal-sede1-mllp.xml.
- **Docker y Docker Compose** (Apache 2.0). Despliegue.

Datos. No se utilizó ningún dato real de pacientes. Los documentos, nombres, diagnósticos y resultados de laboratorio que aparecen en el repositorio, en las evidencias y en la demostración son sintéticos y se generaron para este trabajo. Las claves de los usuarios de demostración están en infra/bus-schema.sql, son sintéticas y solo valen para el entorno

local; los secretos reales se leen de variables de entorno y el archivo .env está excluido del repositorio.

Comandos de la demostración

- docker compose up -d — levanta la plataforma completa.
- bash tests/enviar-las-tres-sedes.sh — ingesta desde las tres sedes en tres formatos.
- python3 tests/prueba-roles.py — accesos permitidos y denegados por rol.
- python3 tests/prueba-vistas.py — las cuatro vistas y el filtrado de datos clínicos.
- python3 tests/verificar-auditoria.py — verificación de la cadena de hash.
- python3 tests/prueba-degradacion.py — caída y recuperación de sedes.
- python3 tests/prueba-normalizador.py — 43 pruebas unitarias del normalizador, sin necesidad de levantar nada.
- bash tests/coleccion-curl.sh — la coleccion de peticiones, numerada por requisito.

AVANCE INICIAL

- Revisión de estándares de interoperabilidad completada: HL7 v2.x con transporte MLLP, FHIR R4 y las terminologías CIE-10 y LOINC.
- Modelado UML terminado: casos de uso con cinco actores y doce casos, secuencia, clases con trece entidades y estados. Generados por script a partir del diseño, de modo que se regeneran sin herramientas externas.
- Entorno completo desplegado con un solo comando: tres nodos PostgreSQL 15 fragmentados, servidor HAPI FHIR R4, Mirth Connect 4.5.2 y el bus de integración, todos en contenedores Docker.
- Portal web funcional con las cuatro vistas por rol y el tablero de cobertura, con el control de acceso aplicado en el servidor.
- Bus de integración operativo: ingesta en tres formatos, modelo canónico, transformación a Bundle FHIR, idempotencia por hash de contenido, reintentos con retroceso exponencial y cola de mensajes muertos.
- Seguridad implementada: token JWT firmado con HMAC-SHA256, claves derivadas con PBKDF2, autorización por rol y por pertenencia, y bitácora encadenada por hash con su verificador.
- Pruebas de extremo a extremo ejecutables, una por cada criterio del enunciado, con sus salidas conservadas como evidencia. Repositorio Git con la historia organizada por fases.

VIABILIDAD Técnica: ESCALABILIDAD Y CUMPLIMIENTO NORMATIVO

Esta sección describe la ruta hacia producción, que va más allá del alcance del prototipo. Para que la distinción quede clara: del listado que sigue, el prototipo implementa hoy el despliegue en contenedores Docker, la fragmentación horizontal de PostgreSQL, la observabilidad mediante endpoints /health y /metrics en cada servicio, el registro de eventos de acceso con integridad verificable en la línea del perfil ATNA, y la autorización basada en roles con token firmado. El resto constituye el trabajo pendiente para un despliegue real: orquestación con Kubernetes y escalado automático, replicación en clúster, cifrado en tránsito y en reposo, gestión de consentimientos, pruebas de carga y los procesos de certificación.

Escalabilidad

- Arquitectura de microservicios desplegada en contenedores Docker orquestados con Kubernetes (Minikube para desarrollo; Kubernetes administrado o OpenShift para producción).
- Horizontal Pod Autoscaling (HPA) y Vertical Pod Autoscaling (VPA) configurados para ajustar recursos dinámicamente según métricas de CPU/RAM y throughput de peticiones FHIR.
- Balanceo de carga mediante Ingress Controller (NGINX o Traefik) y distribución de tráfico basada en rutas y versiones (canary releases/blue-green deployments).
- Base de datos PostgreSQL configurada con particionado lógico y replicación en clúster (Patroni o Aurora PostgreSQL) para alta disponibilidad y escalado read-replica.
- Observabilidad con Prometheus, Grafana y Jaeger para medir latencia, tasa de errores y trazabilidad de transacciones clínicas.
- ~~Pruebas de estrés programadas con JMeter + k6 para validar escalado hasta 10 000 transacciones FHIR por minuto.~~

Cumplimiento normativo

- Alineación con la Ley 1581/2012 (Protección de Datos Personales en Colombia), GDPR (cuando aplique) y directrices de la Superintendencia Nacional de Salud.
- Cifrado extremo-a-extremo: TLS 1.3 en tránsito; cifrado AES-256 at-rest para BD y backups.
- Implementación del perfil ATNA (Audit Trail and Node Authentication) de HL7 para registro de eventos de acceso y auditoría clínica.
- Gestión de consentimientos mediante el recurso FHIR **Consent**, con firmas electrónicas y revocación según normativa.
- Autenticación y autorización basada en OAuth 2.0 / OpenID Connect con Keycloak y políticas RBAC por rol clínico.
- Planes de respaldo y recuperación ante desastres con RPO ≤ 15 min y RTO ≤ 1 hora. Ensayos semestrales.
- Objetivo de certificación ISO 27001 e ISO 27799 (seguridad de la información en salud) en fase de operación.

INTEROPERABILIDAD BASADA EN DATOS: ENFOQUE Semántico, Técnico Y ORGANIZACIONAL

Igual que en la sección anterior, conviene separar lo implementado de lo propuesto. El prototipo cubre hoy: el modelo de datos canónico con los recursos Patient, Encounter, Condition y Observation; las reglas de validación previas a la ingesta; el mapeo terminologico a CIE-10 y LOINC mediante una tabla de correspondencias propia; la exposición de servicios REST con FHIR en JSON; y el enrutamiento de mensajes HL7 v2.x hacia FHIR a través de Mirth Connect. Lo demás describe la evolución natural del sistema hacia un entorno productivo con servidor de terminología, perfiles locales publicados, mensajería asíncrona y gobernanza organizacional.

La interoperabilidad en salud no se limita al intercambio de mensajes; requiere garantizar que los datos circulen con significado, integridad y valor a lo largo de todo el ecosistema clínico-administrativo. A continuación se profundiza en los pilares de interoperabilidad

basada en datos que sustentan la propuesta: calidad, semántica, técnica y gobernanza organizacional.

1. Calidad y Gestión de Datos

- Modelo de datos canónico (FHIR R4) con perfiles locales para **Patient**, **Encounter**, **Observation** y **CarePlan**.
- ~~Estrategia de **Master Data Management (MDM)**: reconciliación de identidades de paciente usando algoritmo probabilístico (Fellegi-Sunter) y servicio PIX/PDQ.~~
- ~~Reglas de validación (JSON Schema + FHIR Validation Module) para garantizar completitud, consistencia y unicidad antes de la ingestión.~~
- ~~Metadatos FAIR (Findable, Accessible, Interoperable, Reusable) mediante etiquetas Dublin Core y recurso **Provenance**.~~
- ~~Versionamiento de recursos con historial (_history) y control de cambios para auditoría y trazabilidad.~~

2. Interoperabilidad Semántica

- Uso de terminologías estandarizadas: **SNOMED CT** (procedimientos y diagnósticos), **LOINC** (laboratorios), **ICD-10**, **RxNorm**.
- Mapeo terminológico automático con **FHIR ConceptMap** y motor de razonamiento (HAPI Terminology Server).
- Perfiles FHIR locales con **ValueSet** vinculados a sistemas de terminología y restricciones business-specific (FHIR Shorthand + IG Publisher).
- Aplicación de reglas de inferencia clínica mediante **Clinical Quality Language (CQL)** para desencadenar alertas y CDS Hooks.
- Validación semántica continua en pipeline CI/CD usando **SUSHI** y tests de terminología.

3. Interoperabilidad Técnica

- Exposición de servicios RESTful FHIR (JSON) y soporte opcional para XML y gRPC FHIR-Proto.
- Implementación de eventos clínicos en tiempo real mediante **FHIR Subscriptions** y mensajería asíncrona (Kafka).
- Compatibilidad con IHE perfiles ITI-41/42 (XDS.b) para documentos clínicos y MHD (Mobile access to Health Documents).
- Enrutamiento de mensajes HL7 v2.x hacia FHIR con **Mirth Connect** y canal de transformación JSR-223 (Groovy).
- Pruebas de conformidad con **Touchstone** y plan de certificación Argonaut/IPS donde aplique.

4. Gobernanza Organizacional

- Comité de Datos Clínicos con representación de TI, médicos, enfermería y gestión de calidad.
- **Políticas de data stewardship**: roles de custodio, propietario y consumidor definidos en RACI.
- Modelo de madurez de interoperabilidad (HIMSS EMRAM) para medir progreso y priorizar inversiones.
- Plan de Change Management con metodología ADKAR para adopción interna y capacitación continua.
- KPIs de interoperabilidad (tasa de matches de pacientes, % de registros válidos semánticamente, tiempo medio de integración) revisados trimestralmente.

5. Herramientas de Integración Técnica

- **HAPI FHIR Server** (open-source, Java): Implementación de referencia FHIR R4/R5; alto control y extensibilidad; requiere esfuerzo de hardening y escalado.
- **Smile CDR** (comercial/Open-Core): Plataforma certificada ONC que integra repositorio clínico, Terminology Server y motores de suscripción; soporte empresarial y actualizaciones rápidas; licenciamiento de pago.
- **Mirth Connect / NextGen Connect** (open-source): Motor de integración HL7 v2, X12 y FHIR con transformaciones JavaScript/Groovy; interfaz visual; comunidad amplia; monitorización limitada sin complementos.
- **OpenHIM** (open-source): Mediador de interoperabilidad del proyecto OpenHIE; flujos de orquestación REST/HL7; auditoría centralizada; requiere Node.js y MongoDB.
- **Apache Camel + camel-fhir** (open-source): Framework de enrutamiento y EIP con más de 300 conectores; flexible para ETL y streaming; curva de aprendizaje intermedia.
- **Kong API Gateway + kong-oas-fhir plugin** (open-source/comercial): Pasarela con políticas de rate-limit, OAuth2 y mTLS; scripts declarativos; licencias premium para analíticas avanzadas.
- **Redox** (SaaS): Plataforma iPaaS especializada en salud; normaliza HL7 v2, CDA y FHIR a su modelo JSON; onboarding rápido; dependencias a nube externa.
- **Rhapsody Integrator** (comercial): Motor de integración ampliamente usado en hospitales; conectores arrastrar-soltar; soporte oficial para IHE perfiles; coste elevado.
- **Google Cloud Healthcare API** / **Azure Health Data Services** (cloud native): Repositorio FHIR gestionado con escalado serverless, DICOM y HL7 bridges; vínculo fuerte a proveedor de nube.

6. Herramientas de Integración Semántica

- **Snowstorm** (SNOMED CT Terminology Server, open-source): Soporta RF2, OWL axioms y FHIR $lookup; rápido y liviano; administración mediante REST.

- **Ontoserver** (CSIRO, open-source/comercial): Terminology server escalable con caché inteligente; implementa $expand, $validate-code; plugins para OMOP y HIE integraciones.
- **Apelon DTS** / **B2i Snow Owl** (comercial): Suites robustas para autoría y gestión de terminologías; UI rica y workflows de gobernanza; coste de licencia.
- **HAPI FHIR Terminology Service** (módulo): Integra ValueSet, ConceptMap y terminologías internas; valida y expande códigos dentro del mismo stack HAPI.
- **OHDSI Athena + OMOP vocabulary** (open-data): Diccionarios estandarizados para investigación; útil para mapeo entre vocabularios clínicos-administrativos.
- **FHIR Shorthand (SUSHI) + IG Publisher** (open-source): Herramientas DevOps para crear guías de implementación, perfiles y ValueSets reproducibles en CI/CD.
- **Phast FHIR Tools** (open-source): Conjunto CLI/GUI para validación y transformaciones; útil en pipelines ETL.
- **OpenRefine + FHIR extension**: Limpieza y mapeo de datos tabulados a ValueSets FHIR; gestiona grandes volúmenes no estructurados.
- **LexEVS** (NCI, open-source): API de terminología para grandes vocabularios biomédicos (LOINC, NCI Thesaurus); despliegue complejo.

7. Compatibilidad de Herramientas con Estándares de Interoperabilidad

La siguiente matriz resume el grado de compatibilidad de las principales herramientas de integración analizadas con los estándares más utilizados en salud: HL7 v2.x, HL7 CDA, HL7 FHIR y perfiles IHE (XDS.b, MHD, PIX/PDQ, ATNA). Los valores indicados representan: Sí ✔️ (soporte nativo o plugin oficial), Parcial ◗ (requiere adaptadores o funciones limitadas) y No ✖️ (sin soporte directo).

Herramienta	HL7 v2.x	HL7 CDA	HL7 FHIR	IHE perfiles
HAPI FHIR Server	✖️	✖️	✔️	◗
Smile CDR	✔️	✔️	✔️	✔️
Mirth Connect	✔️	✔️	✔️	◗
OpenHIM	✔️	◗	✔️	✔️
Apache Camel + camel-fhir	✔️	◗	✔️	◗
Kong API Gateway	✖️	✖️	✔️	✖️
Redox (SaaS)	✔️	✔️	◗	✖️
Rhapsody Integrator	✔️	✔️	✔️	✔️
GCP Healthcare API / Azure HDS	✔️	◗	✔️	✖️

Snowstorm	✗	✗	✓	✗
Ontoserver	✗	✗	✓	✗
Apelon DTS	✗	✗	◗	✗

Tabla 1. Compatibilidad comparativa de herramientas de integración con los estándares de interoperabilidad en salud.

Observaciones clave:
• Las plataformas nativas FHIR como HAPI y Smile CDR aportan soporte pleno a FHIR, pero su compatibilidad HL7 v2/CDA depende de módulos adicionales.
• Motores enterprise (Rhapsody, Mirth) ofrecen la cobertura más amplia, siendo adecuados para entornos con sistemas legados.
• IHE es crucial para intercambio documental (XDS.b, MHD). Smile CDR y Rhapsody incluyen conectores certificados; OpenHIM sigue flujos OpenHIE basados en IHE.
• Herramientas semánticas (Snowstorm, Ontoserver) se centran en FHIR Terminology; su integración con HL7 v2/CDA debe hacerse vía servicios de terminología.

VALIDACIÓN DE MODELOS UML

Métodos

La validación no se delegó a una herramienta de modelado comercial, porque los diagramas de este proyecto no se dibujaron a mano sino que se generan por script. Eso permitió un método distinto y, para este caso, más útil: **contrastar cada diagrama contra el sistema en ejecución**, que es la única forma de garantizar que el modelo y el código no se separen. Se comprobó que los nombres de los recursos coincidan con los de FHIR R4, que las entidades del diagrama de clases correspondan a tablas y clases reales, que los mensajes del diagrama de secuencia correspondan a llamadas que existen, y que los estados del diagrama de estados sean exactamente los valores que toma la columna correspondiente en la base de datos.

Elemento	**Regla de validación**	**Herramienta**
Caso de uso	Cada caso corresponde a un endpoint existente y a un permiso de la matriz de roles	Revisión contra portal.py y seguridad.py
Clase del dominio	Toda entidad existe como tabla con la misma clave primaria y cardinalidad	Contraste con nodo1.sql y bus-schema.sql
Mensaje de secuencia	Verbo HTTP y ruta correctos; el orden reproduce la cadena real	curl contra el sistema en ejecución
Estado	Transiciones permitidas recibido → validado → transformado → entregado, con las	Consulta SQL sobre mensaje_bus.estado

	ramas rechazado y fallido → muerto	

Tabla 2. Reglas y método empleados para validar cada tipo de diagrama contra el sistema en ejecución.

Resultados

La validación reveló cuatro discrepancias entre el modelo y la implementación, todas corregidas en el código y no en el diagrama. La más significativa fue que el diagrama de secuencia mostraba la creación del Encounter en todos los casos, mientras que el sistema lo omitía cuando no lograba interpretar la fecha de atención: el modelo describía el comportamiento correcto y el código el equivocado. Las otras tres fueron un identificador de Patient que se guardaba como marcador en lugar del valor devuelto por el servidor FHIR, tres nodos que compartian el mismo nombre en las respuestas de estado, y un servicio degradado que se reportaba como caído. En los cuatro casos el diagrama resultó ser la descripción correcta.

Discusión

Validar el modelo contra el sistema en ejecución, y no contra sí mismo, convirtió los diagramas en una herramienta de detección de defectos en lugar de en documentación decorativa. El hallazgo del Encounter ausente lo ilustra bien: era un defecto silencioso, que no producía ningún error visible y que las pruebas funcionales habituales no habrían detectado, porque el mensaje se aceptaba y los demás recursos se creaban con normalidad. Apareció al comparar lo que el diagrama prometía con lo que la base de datos contenía. El coste de mantener los diagramas sincronizados con el código se paga solo cuando se descubre un defecto de esa clase.